Software Architecture for Embedded System

Driver

* **Def:**
Driver is a software layer that allows us to control hardware in an organized and reusable way, without dealing directly with registers every time.

⇒ Instead of writing:

```
PORTA |= (1 << 0);
```

⇒ We Use:

```
DIO_SetPinValue(PORTA, PIN0, HIGH);
```

The driver hides low-level details and provides a simple interface.

⇒ Without Drivers:
- code is hard read.
- Code is reapeated many times.
- Hard to debug.
- Hard to modify hardware connections.

⇒ With drives:
- Code is clear and readable.
- Reusable functions.
- Easy to maintain.
- Easy to update connections (via Config).

* **Abstraction Concept:**
hide internal complexity and providing a simple interface.

⇒ Example:
You use a car without knowing how the engine works.
- Same idea: we use functions instead of registers.

⇒ System Layers:

App (Application Layer)
↑
HAL (High Level Driver)
↑
MCAL (LOW Level Driver)
↑
LIB (Utilities)
↑
Hardware

① LIB Layer:

Utility files used by all layers.

⇒ Example:

STD_TYPES.h $\xrightarrow{ex}$ typedef unsigned char U8;

BIT_MATH.h $\xrightarrow{ex}$ #define SET_BIT(REG, BIT) REG |= (1<< BIT)

② MCAL (Microcontroller Abstraction Layer):

Low Level drivers that interact directly with hardware registers.

⇒ Example: DIO, ADC, TIMER, UART, SPI/I2C ...

→ Responsible for controlling microcontroller peripherals.

③ HAL:

Built ontop MCAL provides easy to use components (High level components)

⇒ Example: LCD, Seven Seg., Keypad, Sensors ...

→ HAL uses MCAL internally.

④ App Layer:

This is where the main program is written.

→ Example:

Read sensor → display value.
Pressed Button → turn on LED.

→ App should not deal directly with registers.

⑤ Driver file Structure:

Driver
- → interface file
- → private file
- → Config file
- → program file

① interface file:

public function used by other layers.

- contine
  - → Fun. prototype.
  - → public macros.

→ Example: void DIO_setPinDirection(u8 port, u8 pin, u8 direction);

② private file:

internal details (not for external use).

→ Containe
- → Register definitions.
- → Internal macros.

→ Example: #define DDRA *((volatile u8*) 0x3A)

③ Config. file:

used to configure the driver.

→ contains
- → pin connections.
- → Modes
- → Options

→ Example:

#define LCD_MOD 4BIT

④ Program File:

Containe actual implementation of functions.

→ Example:

```
void DIO_SetPinDirection (u8 port, u8 pin, u8 direction)
{
    if (direction == Output)
    {
        SET_BIT(DDR, pin);
    }
}
```

Advanteges of This Structure:

① Modularity.
② Reusability.
③ Easy debugging.
④ Easy mintenance.
⑤ Clear code structure.

⇒ So, By Separating the System into layer
(LIB → MCAL → HAL → App)
We Make the System
scalable, reusable, easy to maintain.